Week 17 | Attention as a learned kernel on a wake

The scientific question

Weeks 7.3 and 15 have already used Transformer-based representations and discussed neural operators. This lecture returns to the mechanism: which transformations does attention compute, which invariances can be proved, and which physical interpretations require an experiment? The laboratory uses simulated transverse velocity from the existing cylinder-wake archive. Its purpose is to connect matrix algebra to the observation geometry, not to establish a new CFD benchmark.

By the end, a student should construct Q, K and V with explicit tensor dimensions, reproduce scaled dot-product attention without a library attention layer, demonstrate permutation equivariance, and give a counterexample to the claim that a bright attention weight identifies a physically important sensor. Assessment combines four executable tasks with a short explanation of the counterexample.

From observations to tokens

A snapshot is a 32 by 78 region of the simulated wake. A sensor token contains a sampled value from that numerical field and two nondimensional coordinates. We use the term simulated observation to distinguish it from an experiment. Sixteen sensors produce X with shape B by 16 by 3. Coordinate rescaling is determined by the geometry. Value centering and scaling, when fitted, use development training fields only.

Three separate linear maps transform the same input into queries, keys and values. For one head of width d, Q and K have shape B by N by d and V has shape B by N by d_v. The query describes what a receiving token asks for; the key determines its compatibility with each candidate sender; the value carries the information to aggregate. These are useful computational roles, not physical definitions of pressure, transport or force.

$$A(Q,K,V)=\mathrm{softmax}(QK^{T}/\sqrt{d_k})V$$

The softmax acts along the key axis. Consequently each output is a convex combination of value vectors before the output projection and residual layers. It is not generally a convex combination of velocities: the value projection may mix coordinates and velocity, and later layers can have signed coefficients.

A worked two-sensor calculation

Consider one query q=(1,0), keys k1=(1,0) and k2=(0,1), and scalar values v1=2 and v2=-1. With d_k=2, the scores are approximately (0.7071,0). Row normalization gives weights (0.6698,0.3302), and the weighted result is approximately 1.0094. The weights sum to one, but the result changes if V changes while Q and K remain fixed.

The square-root factor follows from a variance calculation. If components of q and k are independent, centered and unit variance, their dot product has variance d_k. Dividing by the square root keeps score variance of order one as head width changes. The assumptions are not guaranteed after training, so this is an initialization argument rather than a theorem about every trained network.

Compute the same example at temperatures 0.1, 1 and 10. Low temperature concentrates mass on the largest score. High temperature approaches a uniform average. A student should check both the numerical result and the derivative behavior; a nearly one-hot softmax can give very small gradients for losing keys.

Permutation equivariance and physical location

Let P be a permutation matrix acting on the sensor axis. Permuting observations and coordinates together changes scores to P Q K^T P^T. Row-wise softmax commutes with this relabeling, so the attended values become P A. Attention is therefore equivariant to token order when token content is permuted consistently. A symmetric pooling operation then makes the final field coefficient vector invariant.

$$A(PQ, PK, PV) = PA(Q, K, V)$$

This statement does not mean that physical position is irrelevant. Swapping two velocities while leaving coordinates fixed changes the observation. Nor does it guarantee generalization to arbitrary new sensor locations. A model trained on sixteen particular locations may still fail when positions move. The release experiment tests dropped sensors and reports that scope explicitly.

The notebook uses a random permutation to test the algebra. A more demanding exercise changes only the coordinates and asks whether the invariance assertion should still pass. It should not: that edit changes the physical input rather than the ordering of an unchanged set.

Attention versus a geometry kernel

A geometry-only Gaussian kernel depends on pairwise coordinate distance. Learned attention instead depends on projected content, which may include the velocity state. Plot both kernels on the same sensor arrangement, and repeat at several fixed frames. A difference between them demonstrates content dependence, not superiority. A frozen random projection is a mechanism demonstration and must not be labeled a trained flow representation.

For a continuum analogy, a weighted sum resembles a discretized integral operator. However, softmax normalization is not an area quadrature rule. On an irregular mesh, physical integration requires cell weights or a carefully specified measure. Likewise, the residual stream and nonlinear feed-forward block move the complete Transformer beyond the interpretation of one linear integral kernel.

$$u'(x_i) \approx \sum_j K(x_i, x_j, u_j)\, v_j$$

The approximation sign matters: this expression motivates a comparison of representations; it does not identify this classroom model with Transolver or another published neural operator. Week 15 remains the course reference for those architectures.

Why weights are not explanations

If every value vector equals the same vector c, every row-normalized attention matrix produces c at every output token. The weight map may be strongly nonuniform, yet its variation has no effect on that output. This is an executable counterexample to a simple attention-as-importance interpretation.

A physical influence study instead needs an intervention contract: what is perturbed, which other quantities stay fixed, which output is measured, and how far the perturbed input lies from training support. Sensor ablation can change both information content and input distribution. Gradient sensitivity is local and depends on normalization. None of these measurements alone establishes fluid-mechanical causation.

Laboratory and assessment

The worked example builds Q, K and V directly from three actual CFD frames. The four coding tasks implement scaled attention, an explicit-callable permutation test, temperature scaling and a masked attention investigation. The constant-value counterexample remains part of the written interpretation. The student notebook leaves those functions incomplete and reports unsubmitted work. The instructor notebook contains separate solutions and runs every assertion in a fresh kernel.

Submit tensor dimensions, two kernel plots, the four passing checks, and a paragraph describing one physical statement the plots do not justify. Code correctness earns 40%, dimensional and normalization reasoning 25%, experimental interpretation 25%, and reproducibility 10%. A visually attractive heatmap cannot substitute for the counterexample or the invariance test.

Reading and continuation

Vaswani et al., Attention Is All You Need (2017), https://arxiv.org/abs/1706.03762, supplies the primary Transformer architecture reference. Raschka's openly available implementation exercises, https://github.com/rasbt/LLMs-from-scratch, are optional additional practice. The derivations and exercises here are independently authored.

Next week changes the token axis from a spatial observation set to an ordered sequence. Permuting time is no longer a symmetry of the forecasting problem. Position information and the causal mask then serve distinct purposes, and the loss must expose the distinction.

Printable laboratory worksheet

Task 1: Scaled attention

Implement row-stable scaled dot-product attention without calling lab.attention. Explain the square-root scaling.

Interface contract: Inputs are floating Torch tensors q,k:[...,N,d], v:[...,N,d_v] with identical optional batch axes and matching N (2-D or 3-D). Return a [...,N,d_v] tensor; normalize over keys.

Task 2: Permutation experiment

Write a function returning the largest equivariance error after consistently permuting all sensor tokens.

Interface contract: Inputs q,k:[N,d], v:[N,d_v] are the unbatched (2-D) Task 1 shapes. order is a length-N Torch permutation. attention_fn is an explicit callable returning the attended tensor, not a weights/output tuple. Return a Python float: maximum absolute equivariance discrepancy. Do not consult a global attention function.

Task 3: Temperature limit

Implement temperature-scaled weights; verify that large temperature moves them toward uniform weights.

Interface contract: Return a [...,N,N] Torch tensor for q,k:[...,N,d] with identical optional batch axes (2-D or 3-D). Divide scaled scores by the finite temperature; nonfinite or nonpositive temperature must raise ValueError.

Task 4: Build an attention investigation

Implement attention over several temperatures and optional key masks. Return weights, output and mean entropy for each temperature. Reject a mask with an empty row. Then vary the temperature and remove half the keys; plot entropy and output change, explaining why constant values hide changes in attention.

Interface contract: Inputs q:[Nq,d], k:[Nk,d], v:[Nk,d_v] are 2-D floating Torch tensors. temperatures is a nonempty sequence of positive scalars. mask is None or a Boolean tensor broadcastable to [Nq,Nk]: True ALLOWS a key (the opposite of torch MultiheadAttention blocking-mask semantics). Invalid tensor dimensions, mismatched widths/counts, nonpositive temperature or a query with no allowed key must raise ValueError. Return a list in temperature order of dictionaries with keys temperature (float), weights ([Nq,Nk] tensor), output ([Nq,d_v] tensor), entropy (float, mean over queries of natural-log entropy).

Before submitting

Restart the kernel and run the worked examples and completed tasks. Confirm that all four coding checks pass and that no retained data or results file changed. Include the requested interpretation, identify every quantity fitted from training data, and separate test observations from decisions made using validation. Record unresolved budget or representation limits rather than removing inconvenient runs.

Assessment and worked interpretation

Coding: Tasks 1–3 carry 5 points each; the integrated Task 4 carries 25. Derivation (25): derive row normalization and square-root scaling. Interpretation (25): give evidence and limitations. Reproducibility (10): identify inputs, split, checkpoint, environment and every failed seed. Total 100. Passing function checks earns only the coding allocation.

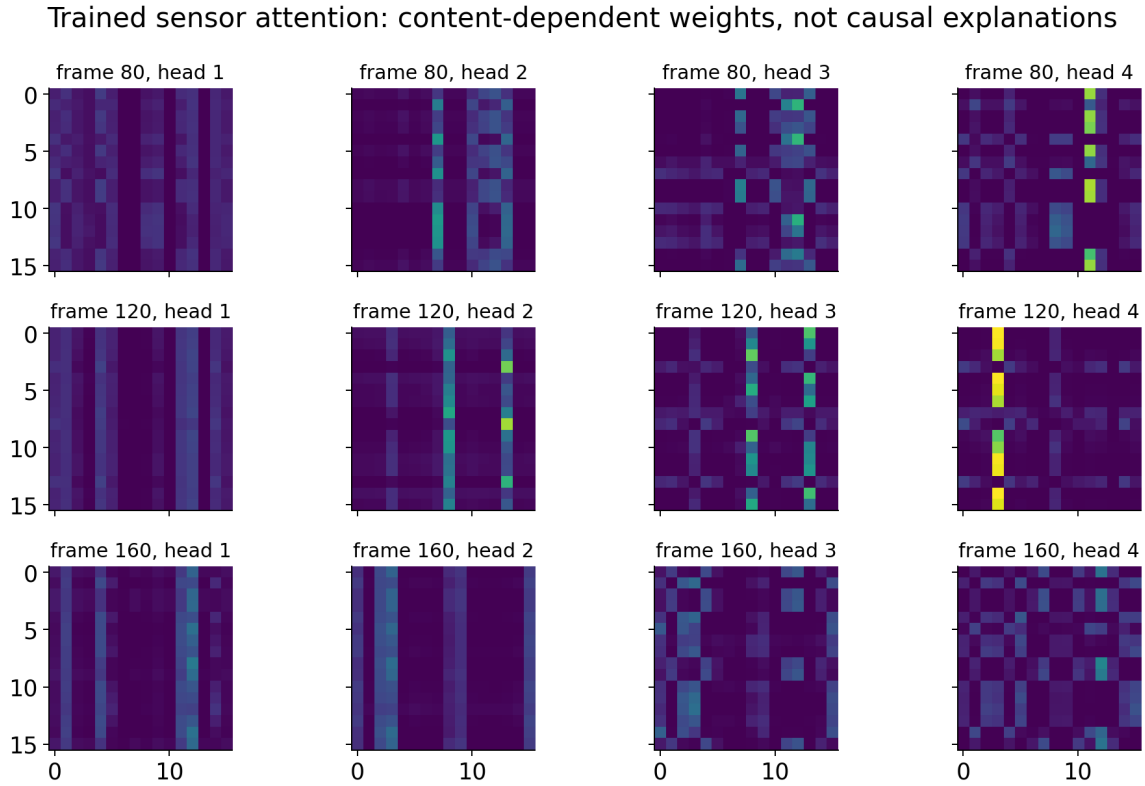
Worked full-credit reasoning: Attention weights depend on content but do not identify causal influence. Equal values make output invariant even when weights change.

Misconception to reject: “The bright head proves this sensor causes shedding.” Explain the failure in one equation or counterexample. An observation without protocol and limitation earns at most half of interpretation credit. Instructor judgment and a timed student pilot remain necessary.

Task 1 and Task 3 accept both [N,d] and [B,N,d] Q/K tensors; values may have a distinct width d_v. Task 2 and Task 4 explicitly use unbatched tensors. Notebook checks cover both declared modes. The trained-head demonstration reads the saved Sensor-set checkpoint (seed 17) at Re=110 frames 80, 120, 160, with width 24 and four width-6 heads.

Timing estimate: a 120-minute guided lab and roughly 3–5 hours including preparation and written analysis. This is an editorial estimate; a timed student pilot remains open.

Executed evidence and interpretation



This figure is generated from the retained protocol. Use the notebook to inspect all seeds and recompute the plotted quantities. The earlier lecture sections specify what the diagnostic can and cannot establish.